온톨로지 정형화 (PDF → JSON) 작업 기록

DLR 온톨로지 사양 v3.00 + FAA JO 7110.65BB → `configs/ontology.json` + `configs/rules.json`

Team 6 (XAI616 Knowledge-Based Systems)

2026-10-15 생성 코드: `src/kb/build_ontology.py` 출력: `ontology.json`, `rules.json` (v1.1)

요약

- 문서 (PDF) 3 개를 근거로, 파이프라인이 읽는 지식 파일 두 개를 **코드로 생성**했다. `configs/ontology.json` 은 **어휘·구조** (무엇을 말할 수 있나, DLR 근거), `configs/rules.json` 은 **규칙** (무엇이 지켜져야 하나, FAA/ICAO 근거와 KG 설계값) 이다 (§6).
- 원칙: **우리가 정한 것 (설계)** 과 **문서가 말하는 것 (근거)** 을 분리하고, 근거로 적은 내용은 **빌드할 때 PDF 원문과 자동 대조**한다. 하나라도 원문에 없으면 빌드가 실패한다.
- 결과: 명령 종류 20 개 모두 검증 통과. 18 개는 DLR 공식 명령에 대응되고, 2 개 (MONITOR, REQUEST) 는 미국 확장이다.
- 파이프라인이 실제로 쓰는 값 (복창 필수 목록, 대체 규칙 등) 은 이전 v0.1 과 동일하다. KATL 전체 KG 를 다시 만들어 그래프와 경고가 똑같음을 확인했다 (§7).

Contents

1 입력 자료 **2**

2 전체 흐름 **2**

3 단계별 방법 **2**

3.1 설계값 정의 (사람이 정함) . . . 2
3.2 DLR 명령 표 파싱 (Table 1 쪽) . . . 3
3.3 DLR 매개변수 표 파싱 (Table 2 쪽) . . . 3
3.4 FAA 조항 분할과 표준 문구 탐색 . . . 3
3.5 검증 규칙 (하나라도 실패하면 생성 중단) . . . 3
3.6 출력 조립: 두 파일로 나누기 . . . 3

4 FAA 복창 등급 **4**

5 매핑 결과 (20 개 종류) **4**

6 두 파일로 나눈 이유와 구조 **5**

7 호환성 확인 **6**

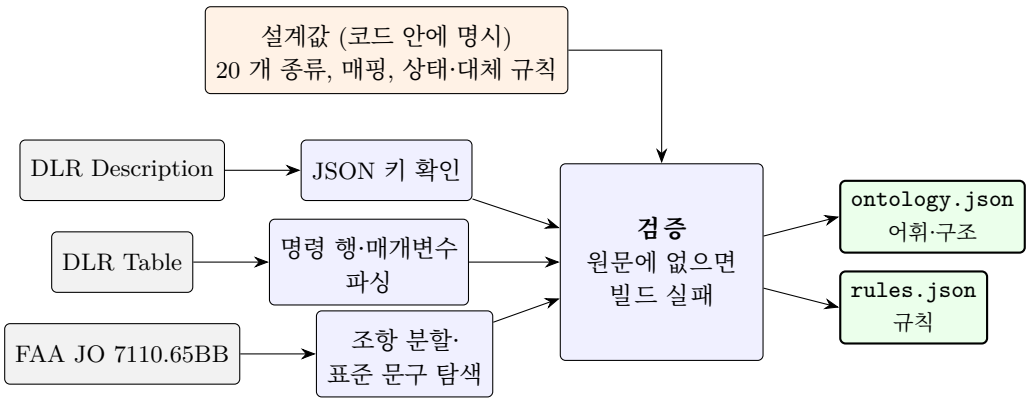
8 재현 방법 **6**

1 입력 자료

세 파일은 `data/ontology/`에 둔다. 저장소에는 올리지 않는다 (`.gitignore`). DLR 문서에는 재배포 제한 문구가 있고, FAA 파일은 9 MB 이기 때문이다. 대신 출력 JSON 의 `generated.inputs` 에 각 파일의 MD5 를 기록해 어떤 판으로 만들었는지 남긴다.

파일	가져오는 내용	출처
`OntologyATCCommands Description_v20250704.pdf` (52 쪽)	지시 (Instruction) 의 구성 요소 (2 장), JSON 교환 키 정의 (5.1 장)	DLR-IB-FL-BS-2025-107 v3.00, https://elib.dlr.de/214852
`OntologyATCCommands Table_v20250704.pdf` (2 쪽)	1 쪽: 16 개 그룹 (제목 줄), Type (Main) 91 개, 126 줄의 Type (Main/Sub), 값, 단위, 한정어, 조건. 2 쪽: 값 형식 (매개변수)	위와 같은 elib 기록의 첨부
`FAA 규정.pdf` (732 쪽)	명령별 표준 문구 (phraseology) 와 조항, 복창 규정 (§2-4-3)	FAA Order JO 7110.65BB (2025-02-20 시행)

2 전체 흐름



모든 PDF 는 `pdftotext -layout`(poppler) 으로 텍스트로 바꾼다. `-layout` 은 표의 열 위치를 공백으로 보존하므로, 글자의 **가로 위치**로 어느 칸인지 판단할 수 있다.

3 단계별 방법

3.1 설계값 정의 (사람이 정함)

코드 맨 위 `COMMAND_TYPES` 에 20 개 종류를 적는다. 종류마다 적는 항목은 다음과 같다.

- `dlr`: 대응하는 DLR 표의 (Type Main, Type Sub). 예: `CLEARED_FOR_TAKEOFF` → (`CLEARED`, `TAKEOFF`). Sub 칸이 비어 있으면 `None`, 아무 Sub 나 허용하면 `*`.
- `faa`: (조항 번호, 그 조항에 있어야 할 표준 문구의 정규식). 예: `HOLD_SHORT` → (3-7-2, `READ BACK HOLD INSTRUCTIONS`). 검증 후 `ontology.json` 의 `faa_phraseology` 로 저장된다.
- `faa_readback`: FAA 복창 등급 (§4). `icao_readback`: ICAO 기준 복창 여부. 둘 다 `rules.json` 으로 간다.
- KG 용 설계값: `state`(유효 구간을 가지는지), `targets`, 대체 규칙, 최대 유효시간, 오탐 방지 키워드 (`lexical_cues`). 이 값들은 규정이 아니라 설계값이라고 JSON 의 `scope_notes` 에 밝힌다.

3.2 DLR 명령 표 파싱 (Table 1 쪽)

1. 머리글 줄에서 각 열 제목 (`Type (Main)`, `Type (Sub)`, `Value(s)`, `Unit`, `Qualifier(s)`, `Condition(s)`) 의 시작 위치를 잰다.
2. 각 줄을 "공백 2 칸 이상" 으로 나눈 조각으로 자르고, 조각의 시작 위치로 열을 정한다. 줄마다 1–2 글자씩 밀리는 경우가 있어 3 글자 여유를 둔다.
3. `Taxi types` 처럼 "types" 로 끝나는 한 조각 줄은 **분류 (category)** 로 기록한다.
4. Main 칸이 빈 줄은 칸 안에서 **줄바꿈된 글자**다. 표가 여러 줄 칸을 세로 가운데 정렬하므로 다음 행에 합친다. 예: `4-digit-squawk-value,` 줄이 아래의 SQUAWK 행 값으로 합쳐진다.
5. 결과: 명령 행 126 개, 그룹 제목 줄 16 개, 서로 다른 Type (Main) 91 개. 이 결과를 `dlr_catalog` 에 그대로 저장한다. 그룹은 **표의 제목 줄 기준 16 개**로 센다. 설명서 9 쪽은 Cleared types 와 Cancel types 를 "Cleared and Cancel types" 하나로 묶어 15 개로 나열하므로, 각 그룹에 설명서 번호 (`description_class`) 를 함께 적고 설명서 목록에 그 번호와 이름이 있는지 확인한다. 표의 오타 (`Informamtion types`) 는 원문 그대로 둔다.

3.3 DLR 매개변수 표 파싱 (Table 2 쪽)

표의 값 칸에는 `runway`, `TX-Point`, `Dir-Qualifier` 처럼 **매개변수 이름**이 들어 있고, 실제 형식은 2 쪽에 있다 (설명서 9 쪽: 굵은 글씨 = 직접 값, 보통 글씨 = 매개변수, 주황 칸 = 선택 요소). 2 쪽을 같은 방식으로 파싱해 매개변수 60 개를 얻는다. 명령 행에서 쓰인 이름만 찾아 붙인다.

- 원문 표는 굵은 글씨 (직접 값) 와 보통 글씨 (매개변수) 를 구분하지만 텍스트 추출에서 글꼴 정보가 사라진다. 그래서 **2 쪽 매개변수 표에 이름이 있는 단어**를 매개변수로 본다. 예: `APTR_COLOR`, `ATSU` 는 대문자지만 매개변수, `EXPEDITE` 는 표에 없으므로 직접 값.
- 표 자체의 오타 (`Alt-Qualfier`) 나 붙어 있는 괄호 설명 (`GND_DIRECTION(only for VIA)`) 은 소문자가 섞인 단어에 한해 유사도 0.85 이상이면 같은 이름으로 본다.

3.4 FAA 조항 분할과 표준 문구 탐색

1. `3-7-2. TAXI AND GROUND MOVEMENT OPERATIONS` 처럼 "번호. 제목" 형태의 줄을 조항 시작으로 본다. 원문은 하이픈 대신 -(U+2212) 를 쓴다.
2. 목차 줄 (점선 . . .) 과 색인에 반복되는 제목은 버리고, **본문의 첫 등장**만 쓴다. 결과는 675 개 조항이다.
3. 각 종류의 (조항, 문구) 에 대해, **그 조항 본문 안에** 문구가 있는지 찾는다. 있으면 앞뒤 60 자를 인용 (`quote`) 으로 저장한다. 다른 조항에만 있으면 그 조항 번호를 오류 메시지로 알려 준다.

3.5 검증 규칙 (하나라도 실패하면 생성 중단)

검증	내용
DLR 지시 구조	Figure 2-1 의 요소 이름 (Callsign, Speaker, Reason, Type, Value(s), Unit, Qualifier, Condition, Conjunction, Requirement), 특수값 (`NO_CALLSIGN`, `NO_CONCEPT`, UNTIL/WHEN/IF/AFTER 등), JSON 키 (`csgn`, `type`, `sndT`, `valu`, `vare`, `qual`, `unit`, `speaker`, `reas`, `cond`) 32 개가 Description 원문에 있는가
DLR 명령	매핑한 (Main, Sub) 행이 표에 실제로 있는가
FAA 조항	조항 번호가 실제 본문 제목으로 있는가, 그 본문에 표준 문구가 있는가
FAA 복창 등급	각 등급의 근거 문장이 §2-4-3 본문에 있는가

3.6 출력 조립: 두 파일로 나누기

검증을 통과한 정보와 설계값을 성격에 따라 두 파일로 나눠 저장한다 (§6). 규칙 파일이 정의되지 않은 명령 종류를 가리키면 생성을 멈춘다. 파이프라인 코드가 읽는 키는 이름과 의미를 그대로 유지하고, 읽는 파일만 바뀌었다.

4 FAA 복창 등급

FAA §2-4-3(PILOT ACKNOWLEDGMENT/READ BACK) 을 읽어 보면, 복창 요구의 강도가 지시마다 다르다. 그래서 "복창 필수 여부" 를 한 비트가 아니라 **등급**으로 기록했다.

등급	원문 근거 (§2-4-3)	해당 종류
`acknowledge`	"Ensure pilots acknowledge all Air Traffic Clearances and ATC Instructions" (모든 지시는 응답 필요)	전체
`mandatory`	"Ensure the read back of hold short instructions"	HOLD_SHORT
`runway_assignment`	"Initial read back of a taxi, departure or landing clearance should include the runway assignment"	TAXI_VIA, CLEARED_FOR_TAKEOFF, CLEARED_TO_LAND
`numbers`	"pilots should read back the 'numbers' "	CONTACT, MONITOR, CLIMB, DESCEND, HEADING, SQUAWK
`design`	규정에 명시 없음. 활주로 진입이나 이동 정지를 허가하는 지시라서 우리가 필수로 정함	CROSS_RUNWAY, LINE_UP_AND_WAIT, HOLD_POSITION

`rules.json` 의 `readback_required.FAA` 는 등급이 있는 13 개 종류이고, v0.1 과 같은 목록이다. ICAO 목록은 입력 파일에 ICAO 문서가 없어서 **검증하지 못했다**고 `readback_basis.ICAO` 에 명시했다. 종류별 등급은 `readback` 에 있다.

5 매핑 결과 (20 개 종류)

우리 종류	DLR (Main Sub)	FAA 조항 (검증된 표준 문구)	FAA 복창 등급
TAXI_VIA	TAXI Preposition (VIA/TO)	3-7-2 "taxi clearance/route must include the specific route"	runway_assignment
HOLD_SHORT	HOLD_SHORT	3-7-2 "READ BACK HOLD INSTRUCTIONS", 2-4-3	mandatory
CROSS_RUNWAY	CROSS	3-7-2 "CROSS (runway)"	design
HOLD_POSITION	HOLD_POSITION	3-7-2 "HOLD POSITION"	design
GIVE_WAY	GIVE_WAY	없음 (표준 문구 없음, 신호등 설명에만 등장)	–
FOLLOW	FOLLOW	3-7-2 "FOLLOW (traffic)"	–
CONTACT	CONTACT, CONTACT_FREQUENCY	2-1-17 "CONTACT (facility name …)"	numbers
MONITOR	없음 *(미국 확장)*	2-1-17 예문 "Monitor Tower", "Monitor Ground"	numbers
LINE_UP_AND_WAIT	LINEUP	3-9-4 "LINE UP AND WAIT"	design
CLEARED_FOR_TAKEOFF	CLEARED TAKEOFF	3-9-10 "CLEARED FOR TAKEOFF"	runway_assignment
CLEARED_TO_LAND	CLEARED LANDING	3-10-5 "CLEARED TO LAND"	runway_assignment
TURN	TURN	3-10-9 "TURN LEFT/RIGHT (taxiway/runway)"	–
CLIMB	CLIMB	4-3-2 "CLIMB AND MAINTAIN"	numbers
DESCEND	DESCEND	4-5-7 "CLIMB/DESCEND AND MAINTAIN"	numbers
HEADING	HEADING (2 개 행)	5-6-2 "TURN LEFT/RIGHT HEADING"	numbers
SQUAWK	SQUAWK	5-2-1 "SQUAWK (code)"	numbers
TRAFFIC_INFO	INFORMATION TRAFFIC	2-1-21 "TRAFFIC, (number) O'CLOCK"	–
WAKE_ADVISORY	CAUTION WAKE_TURBULENCE	2-1-20 "CAUTION WAKE TURBULENCE"	–
ACK	AFFIRM (일부만 대응)	2-4-3 "Wilco," "Roger,"	–

우리 종류	DLR (Main Sub)	FAA 조항 (검증된 표준 문구)	FAA 복창 등급
REQUEST	명령 아님: Speaker=PILOT, Reason=REQUEST	–	–

6 두 파일로 나눈 이유와 구조

지식 기반 시스템은 보통 **온톨로지 (어휘·구조)**, **규칙 베이스**, **추론 엔진**으로 나뉜다. "HOLD_SHORT 라는 명령이 있고 값으로 활주로가 온다" 는 어휘·구조이고, "HOLD_SHORT 는 복창을 반드시 받아야 한다" 는 규정에서 온 규칙이다. 두 정보는 근거 문서도, 쓰는 모듈도 다르므로 파일을 나눴다.

파일	키	근거	읽는 모듈
`ontology.json`	`entity_types`, `relation_types`, `dlr_instruction_schema`, `dlr_catalog`, `command_types`(`state`, `targets`, `description`, `dlr`, `faa_phraseology`), `advisory_types`, `non_instruction_types`, `lexical_cues`	DLR 설명서·표, FAA 표준 문구 (미국에서 쓰는 말인지)	M4 추출, M6 그래프
`rules.json`	`region_rules`, `readback`(종류별 등급), `readback_required`, `readback_basis`, `supersedes`, `max_validity_sec`, `runway_state_types`	FAA §2-4-3, ICAO(미검증), KG 설계값	M6 복창 연결, M7 경고 규칙, E3

이렇게 나누면 RQ4(한국·일본) 에서 **온톨로지는 그대로 두고 규칙만 ICAO 로 바꾼다**는 실험 조건을 파일 단위로 보여 줄 수 있다. 두 파일 모두 `generated` 에 생성 코드, 날짜, 입력 PDF 의 MD5 를 기록한다.

`dlr_instruction_schema`: DLR 지시 구조

설명서 Figure 2-1(7 쪽) 의 구조를 그대로 중첩해 담았다. 각 요소에는 한 문장 설명, JSON 키 (5.1 장, 20 쪽), 우리 M4 필드 (`ours`), 필수 여부와 빈 값, 우리와 다른 점 (`note`) 을 적었다.

묶음	요소	JSON 키	원문 규칙 → 우리 필드
–	Callsign	`csgn`	필수, 없으면 `NO_CALLSIGN` → `callsign`
Command	Speaker	`speaker`	ATCO/PILOT, 기본 ATCO → `speaker` (송신 단위로 한 번)
	Reason	`reas`	REQUEST/REPORTING, 조종사 발화만 → 없음 (요청은 명령 종류 REQUEST 로 단순화)
	Type	`type`, `sndT`	필수, 없으면 `NO_CONCEPT` → `type` (Main+Sub 를 이름 하나로)
	Value(s)	`valu`, `vare`	→ `target` / `value` / `route`
	Unit	`unit`	→ 없음 (지상관제에서 거의 안 씀)
	Qualifier	`qual`	→ `qualifier`
Condition(s)	Conjunction	`cond`	UNTIL/WHEN/IF/AFTER → 없음 (범위 제외)
	Requirement		예: 5 NM FINAL → 없음

예시 (HOLD_SHORT, 일부 생략):

```
ontology.json
"HOLD_SHORT": {
  "state": true, "targets": ["Runway", "Taxiway"],
  "dlr": [{"category": "Ground hold types", "main": "HOLD_SHORT",
           "value": "runway/TX-Point/Alphabet-Letter",
           "parameters": {"runway": "runway_without_none + \"none\"",
                          "TX-Point": "Name of defined Taxi-Point ... such as A (alfa), B1, HP_05R"}}],
  "faa_phraseology": [{"section": "3-7-2", "title": "TAXI AND GROUND MOVEMENT OPERATIONS",
                       "quote": "... PHRASEOLOGY- READ BACK HOLD INSTRUCTIONS ..."}, ...]
}
```

```
rules.json
"readback": {"HOLD_SHORT": {"FAA": "mandatory", "ICAO": true}, ...},
"readback_required": {"FAA": ["TAXI_VIA", "HOLD_SHORT", ...], "ICAO": [...]},
"readback_basis": {"FAA": {"mandatory": {"section": "2-4-3",
                    "quote": "... b. Ensure the read back of hold short instructions ..."}, ...}}
```

7 호환성 확인

1. **값 비교:** 두 파일의 값을 이전 단일 파일 (v0.1) 과 비교했다. `readback_required`, `region_rules`, `supersedes`, `max_validity_sec`, `runway_state_types`, `lexical_cues`, `advisory_types`, `non_instruction_types`, `entity_types`, `relation_types`, 20 개 종류의 `state` 와 복창 등급이 모두 같다.
2. **전체 재실행:** 저장된 추출 결과로 KATL 전체 KG(`results/katl_all`) 를 다시 만들었다. 경고 파일 (`alerts.json`) 은 바이트 단위로 같고, 그래프 (`graph.json`) 는 적재 시각 (`ingested_at`) 을 빼면 같다. 노드 42,954 개, 간선 65,016 개, 복창 연결 404 건 (일치 232, 부분 138, 불일치 34) 이다.

8 재현 방법

```
# data/ontology/ 에 PDF 3개를 둔 뒤
python src/kb/build_ontology.py                      # configs/ontology.json, configs/rules.json 생성
python src/kb/build_ontology.py --out-dir /tmp/kb    # 다른 폴더로 시험 생성
```

매핑을 바꾸려면 코드의 `COMMAND_TYPES` 만 고치고 다시 실행한다. 원문에 없는 근거를 적으면 어떤 항목이 왜 실패했는지 출력하고 멈춘다.

9 한계

한계와 주의

- **매개변수 표 줄바꿈:** DLR 표 2 쪽에서 여러 줄로 나뉜 칸 일부가 잘려 들어온다 (예: `altitude` 는 "or none" 만, `4-digit-squawk-value` 는 빈 값). 1 쪽 명령 행은 정확하다.
- **조건·단위 미추출:** DLR 의 Condition(UNTIL/WHEN/IF/AFTER) 과 Unit 은 표에서 옮겨 적기만 했고, M4 추출에서는 아직 쓰지 않는다.
- **ICAO 미검증:** ICAO 복창 목록은 Doc 4444 를 입력으로 넣지 않아 자동 검증 대상이 아니다.
- **TAXI TO/VIA 묶음:** DLR 은 "taxi to A4 via L N7" 을 TAXI TO 와 TAXI VIA 두 Instruction 으로 나누지만 (설명서 8 쪽 예시), 우리 TAXI_VIA 는 하나에 경로 (`route`) 와 목적지 (`target`) 를 함께 담는다. `dlr_instruction_schema` 의 Value(s) 항목에 명시했다.
- **매핑은 사람이 정함:** 우리 종류와 DLR 행의 대응 (예: TAXI_VIA → TAXI VIA/TO) 은 사람이 판단했다. 코드는 그 행이 존재하는지만 확인한다.
- **문구 위치 검증:** FAA 표준 문구가 해당 조항에 "있다" 는 것만 확인한다. 그 조항이 그 지시의 유일한 근거라는 뜻은 아니다.